

OrangeBlossom Database

Term Project 2

Report

Part 1 of Report

Purpose for database:

Orange Blossom Nursery requires a database system to efficiently manage the large amount of information involved in its plant orders, client accounts, billing records, employee specialties, and purchasing activities. Currently, the company collects detailed information through multiple forms, including client order forms, billing statements, and purchase orders. As the business grows and handles more contractors, homeowners, and landscaping firms, storing and organizing information without a structured database would become inefficient, prone to error, and difficult to manage.

One primary reason a database is necessary is because of the complexity of plant orders. Contractors and landscaping firms often place large orders that contain multiple plant varieties, sizes, colors, and quantities. These projects may also be completed in phases and scheduled over long periods of time. A database allows Orange Blossom to store detailed information about each project, including plant specifications, planned locations, installation dates, and pricing. Without a database system, managing information manually would make it difficult for employees to track orders, update project details, and ensure that all the correct plants are available when needed.

A database system also improves Orange Blossom's ability to manage client accounts and billing. Many contractors and landscape designers purchase plants for their own customers and

may make payments over time as the project progresses. The company must track payments, outstanding balances, and project totals while linking them to specific clients and projects. A database makes it possible to store and organize this financial information accurately, while allowing the accounting manager to generate billing statements and monitor client balances Efficiently.

Inventory planning is another important area where a database is beneficial. Because many plants take a long time to grow, Orange Blossom must forecast demand and plan production in advance. By storing historical order data and project information, the company can analyze trends and better estimate how many plants should be grown for future projects. This helps reduce the risk of overproduction, which can lead to unsold inventory and lower profits, or underproduction, which can result in lost sales opportunities.

In addition, the company works with multiple vendors and suppliers to purchase materials. A database system allows Orange Blossom to track purchase orders, vendor locations, delivery dates, and pricing details. This ensures employees can monitor supply levels and coordinate deliveries more efficiently.

Finally, a database can better support employee management by tracking the specialties of each employee. Since employees may have specialties in different areas, such as landscaping or installation, maintaining this information allows a company to assign staff to projects more

Efficiently.

Overall, implementing a database system will improve organizational problems, allow Orange Blossom Nursery to centralize its operational data, improve accuracy, and streamline the management of orders, payments, inventory, vendors, and employees. This structured approach will support the company's growth while helping it maintain its high level of service and organization required to meet the needs of contractors, landscapers, and homeowners

Business Assumptions:

Several business assumptions were made while designing the database structure for Orange Blossom Nursery to support the provided business rules and normalization process. Each project is assumed to be associated with exactly one client and managed by one employee. However a client may have multiple projects over time and an employee may manage many projects. This assumption simplifies the relationship between clients, employees, and projects. In addition, it was assumed that each order form was linked to a single project, even though projects may contain many different design segment items representing different plants. Another assumption relates to plant information. Plants are uniquely identified by an ItemID and include attributes such as species, description and fixed price. However, the price recorded in an order may differ from the plant's fixed price depending on the difficulty of the task or any discounts available at the time of the sale. Therefore, the database stores both the fixed price for the plant and the employee determined sale price within the order line. For the plants, it was assumed that plant characteristics such as size and color may vary depending on the project requirements. These attributes are stored within the order line rather than the plant table itself.

Employees may have multiple specialities and that each specialty can apply to multiple employees. This creates a many-to-many relationship between employees and specialties. This allows the company to assign employees to projects based on their skills and expertise. In addition, it helps the company keep track of what speciality an employee has.

For the billing process, it was assumed that payments are made toward specific projects rather than general client accounts. Each payment includes a payment date and the amount that is linked to a single project. In addition, a project may receive multiple payments over time. The database also assumes that calculated values such as balances, total received, and outstanding balance should be stored because they can be calculated from other stored data.

Finally, it was assumed that vendor purchases are associated with a specific vendor location and processed by one employee. Each purchase order may contain multiple items, and each item has both a fixed price and a potential sale price depending on supplier promotions or market fluctuations at the time of purchase. These assumptions ensure that the database can accurately represent purchasing activity while maintaining normalized data structure and avoiding redundancy information.

Part 2 of Report:

ClientOrderForm Queries

Query 1 Rationale: Each plant at Orange Blossom has a fixed list price, but employees can adjust the actual sale price based on the difficulty of the job, sales availability, or other client-specific circumstances. Calculating the average sale price for each item across all client orders gives management visibility into how items are actually being priced in practice versus their established list prices. This information supports pricing strategy decisions, helps identify

items that are consistently being discounted or up charged, and provides data for forecasting revenue and reviewing whether current list prices remain appropriate given real selling behavior.

Query 2 Rationale: Identifying clients who have placed more than one project order is essential for understanding customer loyalty and recognizing the company's recurring revenue base.

Repeat clients are typically contractors and landscape designers who plan multiple projects with Orange Blossom over time, and these accounts represent the most reliable source of long-term business. This query allows management to flag loyal clients for VIP treatment, volume discounts, or priority scheduling, and it also helps the marketing team distinguish between one-time buyers and high-value relationships that should be nurtured for continued growth.

Query 3 Rationale: Tracking the total quantity of items ordered for each project gives management a clear view of the size and scope of every engagement. This information is important for inventory planning, since large projects may require Orange Blossom to grow or source significant volumes of plants in advance, and for resource allocation, since bigger projects typically require more employees and longer installation times. By calculating the total quantity per project at query time rather than storing it as a computed value, the company stays compliant with the database design rule that no calculated values should be stored, while still giving operations the data they need to plan effectively.

Query 4 Rationale: Identifying clients who have never placed a project order helps the sales and marketing teams uncover untapped opportunities within the existing customer base. These clients are already in the database, meaning they have at some point engaged with Orange Blossom, but have not yet converted into active project customers. This query supports targeted outreach campaigns, follow-up calls, and promotional offers aimed at converting dormant accounts into

paying clients, and it also helps management evaluate the effectiveness of past lead generation efforts by surfacing prospects who have not yet been activated.

BillingForm Queries

Query 1 Rationale:

OrangeBlossom needs to identify its highest-value clients to prioritize account management and follow-up. By filtering clients whose cumulative payments exceed \$500, the company can distinguish major accounts from smaller ones, enabling targeted relationship management and upselling opportunities. This query joins client, billing, and payment data to give a complete financial picture per client.

Query 2 Rationale:

Management needs to evaluate employee billing performance to understand workload distribution and project scale handled per staff member. Knowing the average project cost per employee helps identify which employees are managing larger, more complex projects versus smaller ones. This supports decisions around staffing, compensation review, and workload balancing across the billing team.

Query 3 Rationale:

The company can use this query to flag outlier payments that are significantly above the norm, which may require additional review, documentation, or approval. For example, unusually large payments could indicate high-cost projects that need executive oversight, or could help the finance team audit billing records for accuracy. The subquery dynamically calculates the average so the threshold always reflects current data.

Query 4 Rationale:

Tracking outstanding balances is critical for the company's accounts receivable process. This query gives the billing team a real-time snapshot of how much each client still owes after their payments, as well as how frequently they have been making payments. Clients with high remaining balances and few payments may require follow-up, while consistent payers can be flagged for positive relationship management.

Purchase Order Form Queries

Query 1 Rationale

Understanding where and when purchase orders are placed helps the company monitor vendor relationships and operational activity. By identifying purchase orders after a specific date and linking them to vendor locations, managers can track recent purchasing trends, evaluate supplier performance, and ensure that procurement is aligned with current business needs.

Query 2 Rationale

This query helps assess employee productivity and workload distribution within the purchasing process. By identifying employees who have processed more than one purchase order, management can recognize high-performing employees, ensure tasks are distributed fairly, and detect potential bottlenecks or over-reliance on certain staff members.

Query 3 Rationale

Monitoring how many purchase orders each vendor receives is essential for supplier management and strategic sourcing. This query allows the company to identify which vendors

are most frequently used, evaluate dependency on specific suppliers, and make informed decisions about diversifying vendors or strengthening key partnerships.

Query 4 Rationale

Analyzing items with prices above the average helps the company understand its pricing structure and identify high-value products. This information is useful for inventory management, cost control, and strategic decision-making, as managers can focus on premium items that may contribute more significantly to revenue or require closer monitoring.

Business Scenarios

Scenario 1: End-of-Quarter Billing Review

At the end of each quarter, the OrangeBlossom finance manager conducts a billing review to assess the company's revenue collection status and employee performance. During this review, the manager needs to identify which clients have paid the most overall to prioritize account check-ins (Query 1), understand how each billing employee is performing in terms of project cost handled (Query 2), and flag any unusually large payments that may require documentation or approval (Query 3).

Query Types Required:

INNER JOIN queries with GROUP BY + HAVING (Q1), Aggregate query with GROUP BY (Q2), Subquery with WHERE (Q3).

Scenario 2: Accounts Receivable Follow-Up

The billing coordinator at OrangeBlossom needs to follow up with clients who have outstanding balances at the end of a project cycle. They run a report to see each client's total number of

payments made and their remaining balance (Query 4). Clients with large remaining balances are contacted for payment reminders, while clients who have paid in full are marked as closed.

Additionally, the coordinator cross-references above-average payments (Query 3) to ensure no billing errors occurred on high-cost projects.

Query Types Required:

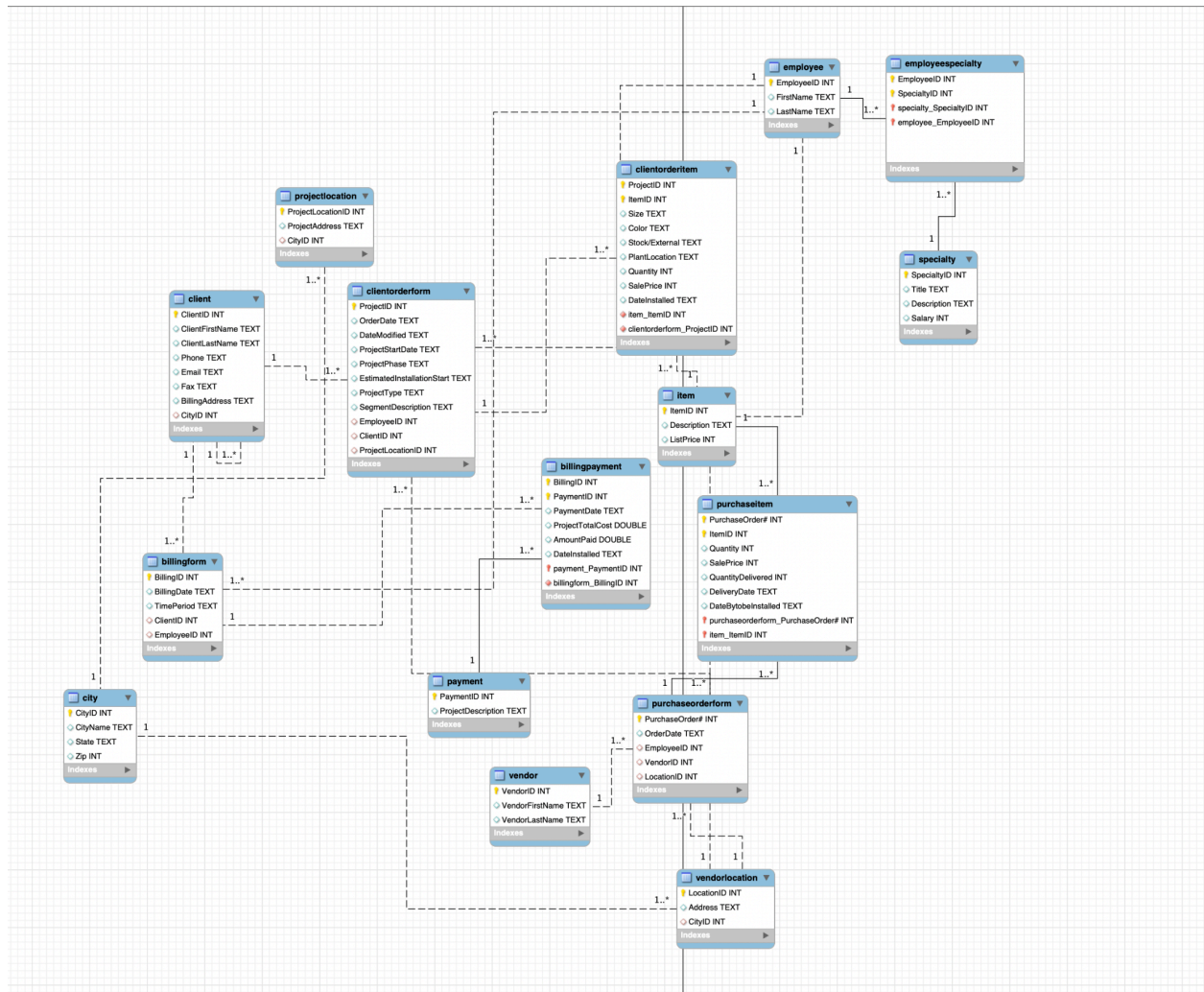
Aggregate queries with GROUP BY (Q4), Subquery with WHERE (Q3).

Appendix A: Normalized Tables

Final Normalized Tables List

1. **ClientOrderForm**(ProjectID, OrderDate, DateModified, ProjectStartDate, ProjectPhase, EstimatedInstallationStart, ProjectType, EmployeeID, ClientID, LocationID)
2. **ClientOrderItem**(ProjectID, ItemID, Size, Color, Stock/External, PlantLocation, Quantity, SalePrice, DateInstalled)
3. **Item**(ItemID, Description, ListPrice)
4. **Employee**(EmployeeID, EFirstName, ELastName)
5. **Client** (ClientID, ClientContactPersonFirstName, ClientContactPersonLastName, ClientPhone, Email, Fax, ClientAddress, CityID)
6. **ProjectLocation** (ProjectLocationID, ProjectAddress, CityID)
7. **City**(CityID, CityName, State, Zip)
8. **EmployeeSpecialty**(EmployeeID, SpecialtyID)
9. **Specialty**(SpecialtyID, Title, Description)
10. **BillingForm** (BillingID, BillingDate, TimePeriod, ClientID, EmployeeID)
11. **BillingPayment** (BillingID, PaymentID, PaymentDate, ProjectTotalCost, AmountPaid, DateInstalled)
12. **Payment** (PaymentID, ProjectDescription)
13. **PurchaseOrderForm** (PurchaseOrder#, OrderDate, EmployeeID, VendorID, LocationID)
14. **PurchaseItem** (PurchaseOrder#, ItemID, Quantity, SalePrice, QuantityDelivered, DeliveryDate)
15. **Vendor**(VendorID, VendorFirstName, VendorLastName)
16. **VendorLocation**(LocationID, Address, CityID)

Appendix B: Class Diagrams



Appendix C: Queries

ClientOrderForm

Michelle Tran

QUERY 1: What is the average sale price per item?

SQL:

```
SELECT item.ItemID, item.Description, AVG(clientorderitem.SalePrice) AS AverageSalePrice
FROM item
INNER JOIN clientorderitem ON item.ItemID = clientorderitem.ItemID
WHERE clientorderitem.SalePrice IS NOT NULL
GROUP BY item.ItemID, item.Description;
```

QUERY 2: Which clients have placed more than one project order?

SQL:

```
SELECT client.ClientID, client.ClientFirstName, client.ClientLastName,
       Count(clientorderform.ProjectID) AS NumberOfProjects
FROM client
INNER JOIN clientorderform ON client.ClientID = clientorderform.ClientID
WHERE clientorderform.ProjectID IS NOT NULL
GROUP BY client.ClientID, client.ClientFirstName, client.ClientLastName
HAVING COUNT(clientorderform.ProjectID) > 1;
```

QUERY 3: What is the total quantity of items ordered for each project?

SQL:

```
SELECT ProjectID, Sum(Quantity) AS TotalQuantity
FROM ClientOrderItem
WHERE Quantity is NOT NULL
GROUP BY ProjectID;
```

QUERY 4: Which clients have never placed a project order?

SQL:

SELECT ClientID, ClientFirstName, ClientLastName

From Client

Where ClientID NOT IN (

 SELECT ClientID

 FROM ClientOrderForm

 WHERE ClientID is NOT NULL

BillingForm

Geyer-Kim, Juliana

QUERY 1: Which clients have a total amount paid exceeding \$500?

SQL:

select client.ClientID, client.ClientFirstName, client.ClientLastName
from client

inner join billingform ON client.ClientID = billingForm.ClientID

inner join billingpayment ON billingForm.BillingID = billingPayment.BillingID

WHERE billingPayment.AmountPaid IS NOT NULL

GROUP BY Client.ClientID, Client.ClientFirstName, Client.ClientLastName

HAVING SUM(billingPayment.AmountPaid) > 500;

QUERY 2: What is the average project total cost per employee who has handled billing?

SQL:

SELECT employee.EmployeeID, employee.FirstName, employee.LastName,

AVG(billingpayment.ProjectTotalCost) AS AvgProjectCost

FROM employee

inner join billingform ON employee.EmployeeID = billingForm.EmployeeID

inner join billingpayment ON billingForm.BillingID = billingPayment.BillingID

WHERE billingpayment.ProjectTotalCost IS NOT NULL

GROUP BY employee.EmployeeID, employee.FirstName, employee.LastName;

QUERY 3: Which billing payments have an amount paid above the average payment amount?

SQL:

SELECT BillingID, PaymentID, AmountPaid, ProjectTotalCost

FROM billingpayment

WHERE AmountPaid > (

 SELECT AVG(AmountPaid)

```
FROM billingpayment
WHERE AmountPaid IS NOT NULL
);
```

QUERY 4: How many payments has each client made, and what is their remaining balance?

SQL:

```
SELECT client.ClientID, client.ClientFirstName, client.ClientLastName,
       COUNT(billingPayment.PaymentID) AS NumberOfPayments,
       SUM(billingPayment.ProjectTotalCost) - SUM(billingPayment.AmountPaid) AS
RemainingBalance
FROM client
inner join billingform ON client.ClientID = billingForm.ClientID
inner join billingpayment ON billingForm.BillingID = billingPayment.BillingID
WHERE billingForm.BillingDate IS NOT NULL
GROUP BY client.ClientID, client.ClientFirstName, client.ClientLastName;
```

Purchase Order Form

Naomi Pisfil-Capunay

QUERY 1: Which purchase orders were placed with each vendor location after January 1, 2024?

SQL:

```
select purchaseorderform.`PurchaseOrder#`, purchaseorderform.OrderDate,
vendor.VendorFirstName, vendor.VendorLastName, vendorlocation.Address
from purchaseorderform
inner join vendorlocation ON purchaseorderform.LocationID = vendorlocation.LocationID
inner join vendor ON purchaseorderform.VendorID = vendor.VendorID
WHERE purchaseorderform.OrderDate >= '2024-01-01';
```

QUERY 2: Which employees processed more than 1 purchase order??

SQL:

```
select employee.EmployeeID, employee.FirstName, employee.LastName,
COUNT(purchaseorderform.`PurchaseOrder#`) AS TotalPurchaseOrders
from employee
inner join purchaseorderform ON employee.EmployeeID = purchaseorderform.EmployeeID
WHERE purchaseorderform.OrderDate IS NOT NULL
GROUP BY employee.EmployeeID, employee.FirstName, employee.LastName
HAVING COUNT(purchaseorderform.`PurchaseOrder#`) > 1;
```

QUERY 3: How many purchase orders has each vendor received?

SQL:

```
select vendor.VendorID, vendor.VendorFirstName, vendor.VendorLastName,  
COUNT(purchaseorderform.`PurchaseOrder#`) AS NumberOfPurchaseOrders  
from vendor  
inner join purchaseorderform ON vendor.VendorID = purchaseorderform.VendorID  
WHERE purchaseorderform.VendorID IS NOT NULL  
GROUP BY vendor.VendorID, vendor.VendorFirstName, vendor.VendorLastName;
```

QUERY 4: Which items have a list price higher than the average list price?

SQL:

```
select item.ItemID, item.Description, item.ListPrice  
from item  
WHERE item.ListPrice > (  
select AVG(item.ListPrice)  
from item  
WHERE item.ListPrice IS NOT NULL  
);
```